# UNIT V

## Managing Console I/O Operations

Console I/O Operations:

1) Programs follow input → process → output cycle.
2) C++ provides advanced I/O functions (via classes, inheritance, virtual functions). Advanced I/O" refers to the C++ doesn't just give you cin and cout, it gives you a whole framework of classes that you can extend and customize.

E.g:

a) **Classes**: **Input/output in C++ is handled by classes** *like istream, ostream, ifstream, ofstream*. **These are objects that represent streams of data.**

Program 1: Write a Program using Input & Output Stream in C++

```
#include <iostream>
// This line includes the standard input/output stream library.
// It gives access to objects like cout, cin, and classes like ostream.

using namespace std;
// This allows you to use names from the standard library (like cout, string)
// without writing std:: before them.

class MyOutput : public ostream
// Defines a new class called MyOutput.
// It inherits (extends) the standard output stream class 'ostream'.
// This means MyOutput is treated like an output stream.

{
public:
    // Public section: members here can be accessed from outside the class.
```

```
    void writeCustom(const string& s)
    // Defines a function named writeCustom that takes a string reference as input.
    // 'const' means the string cannot be modified inside the function.
    {
        cout << "[Custom Output] " << s << endl;
        // Prints the string to the console, prefixed with "[Custom Output]".
        // 'endl' inserts a newline and flushes the output buffer.
    }
};

int main() {
    // The main function: program execution starts here.

    MyOutput myOut;
    // Creates an object 'myOut' of type MyOutput.

    myOut.writeCustom("Hello World");
    // Calls the writeCustom function on 'myOut'.
    // This prints: [Custom Output] Hello World
}
```

**b) Inheritance**: These classes are arranged in a hierarchy. For example:

- istream (input stream) and ostream (output stream) both inherit from ios.
- File streams (ifstream, ofstream) inherit from these to add file-specific behavior.

c) V**irtual functions**: Many I/O operations are defined as virtual functions, which allows polymorphism. This means you can override behavior in derived classes. For example, you can create a custom stream class that formats output differently but still works with << and >>.

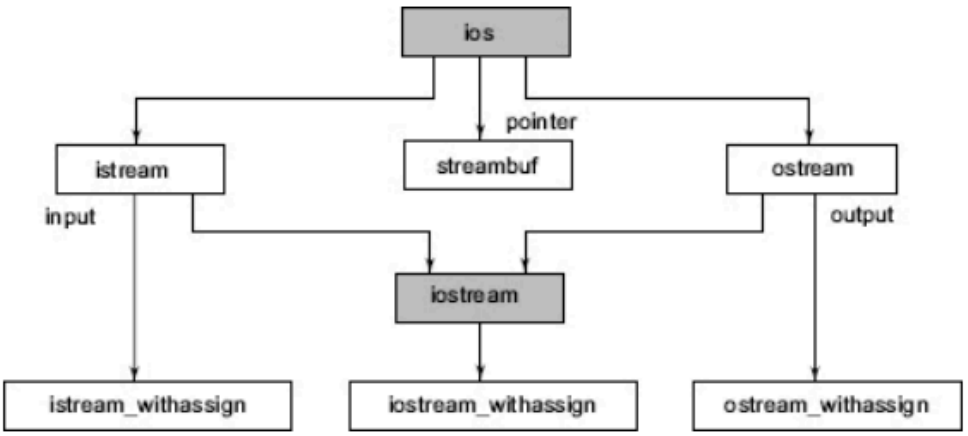
3) Unlike C, C++ I/O supports OOP concepts and user-defined data types.

Uses streams and stream classes for console and file I/O.
cin → standard input (keyboard), cout → standard output (screen).

## C++ Streams

Stream = sequence of bytes; acts as interface between program and device.
Input stream: source of data (keyboard, file).
Output stream: destination for data (screen, file).
Predefined streams: cin, cout.
Streams can be redirected to other devices/files.

## C++ Stream Classes




Remember **hierarchy**: ios → istream/ostream → iostream.
Know **operators**: >> (extraction), << (insertion).
**Manipulators**: endl, setw, setprecision are part of advanced formatting.
**File I/O**: ifstream, ofstream, fstream extend console streams to files.

**Polymorphism**: Virtual functions allow overriding stream behavior.
**Buffering**: streambuf is the hidden engine behind streams.

10.4 Unformatted I/O Operations

**Formatted I/O in C++**

Formatted I/O uses manipulators and stream operators to control how data is read or displayed.

- **Functions/Objects used:** cin, cout, along with manipulators like setw, setprecision, fixed, etc.

**Unformatted I/O in C++:**

Unformatted I/O deals with raw character or string input/output, without format specifiers or manipulators.

- **Functions used:** get(), put(), getline(), write(), read()

>> (in istream) → input.
<< (in ostream) → output.

cin >> var1 >> var2;

- cin is the standard input stream (usually the keyboard).
- The operator >> is called the **extraction operator**.
- It reads values from input and stores them into variables (var1, then var2).
- Input is separated by **whitespace** (spaces, tabs, newlines).
- Example: If you type 10 20, then var1 = 10 and var2 = 20.

cout << item1 << item2;

- cout is the standard output stream (usually the console).
- The operator << is called the insertion operator.
- It sends values to the output stream.
- Here, it prints item1 followed immediately by item2.
- Example: If item1 = "Hello" and item2 = "World",
- output will be: Hello World

## get() and put() Functions

cin.get(c) → reads single character (including spaces, tabs, newline).
cin.get() → returns character directly.
cout.put(ch) → outputs single character.

Example:

```
#include <iostream>
// This includes the standard input/output library so we can use cin, cout, etc.
using namespace std;
// Lets us use names like cout and cin directly without writing std::cout.
int main()
// The main function — program execution starts here.
{
    int count = 0;
    // A variable to keep track of how many characters are entered.
    char c;
   // A variable to store one character at a time.
    cout << "INPUT TEXT\n";
   // Prints the message "INPUT TEXT" and moves to a new line.
    cin.get(c);
   // Reads the first character typed by the user (including spaces).
    while(c != '\n')
   // Loop continues until the user presses Enter (newline character).
    {
        cout.put(c);
      // Prints the character back to the screen.
        count++;
      // Increases the count by 1 for each character.
        cin.get(c);
```

```
    // Reads the next character from input.
  }
  cout << "\nNumber of characters = " << count << "\n";
  // After the loop ends, prints the total number of characters entered.
  return 0;
// Ends the program successfully.
}
```

Output:

INPUT TEXT

Hello World
Hello World
Number of characters = 11